

CAHIER DES CHARGES

Mini-projet Full Stack : Développement d'une plateforme E-Store

Module : Full Stack

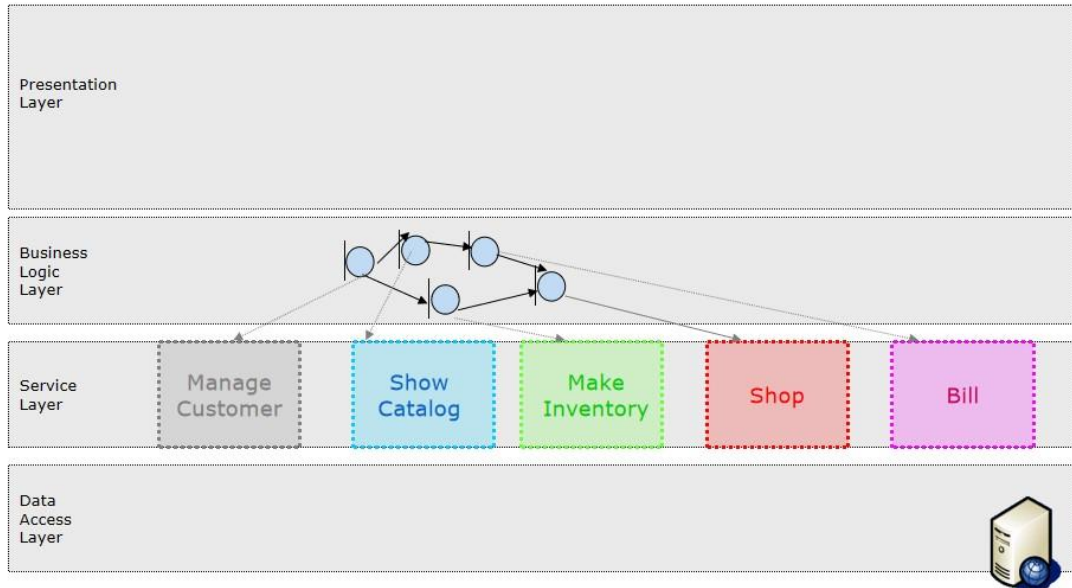
Technologies imposées : Spring Boot, Spring Data JPA, MongoDB, Angular, Maven

Encadrant : Pr. Omar Zahour

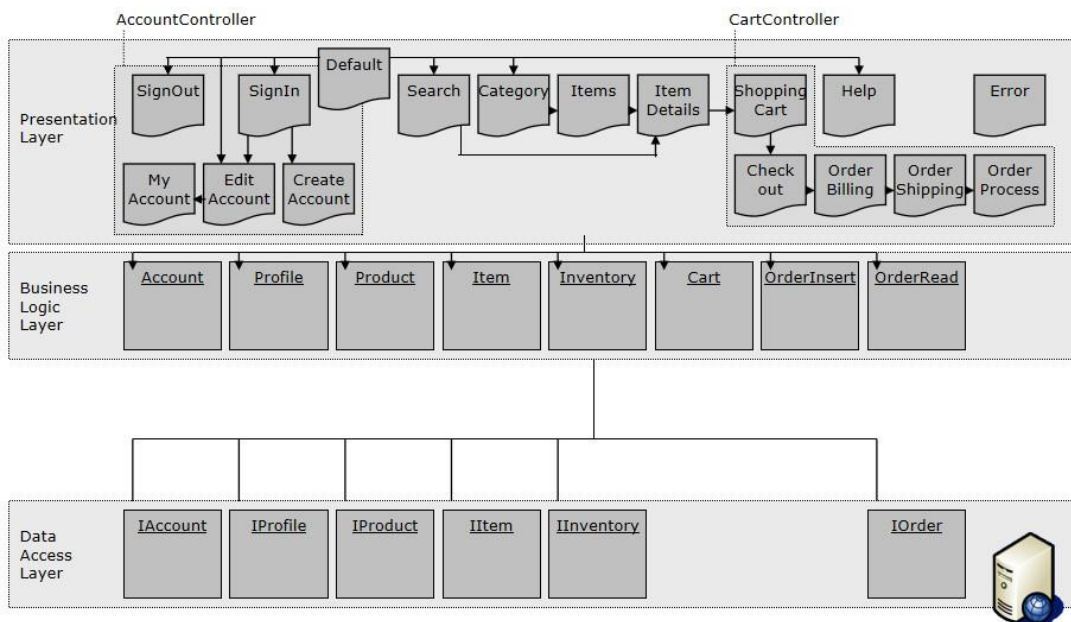
Faculté des Sciences Ben M'Sick - Université Hassan II de Casablanca

Nature du livrable	Document de référence obligatoire pour la réalisation du mini-projet
Type de projet	Application e-commerce simplifiée, structurée en couches et en domaines
Travail demandé	Analyse, conception, développement, tests, rapport et soutenance
Résultat attendu	Une application full stack propre, fonctionnelle, documentée et démontrable

Exemple d'un e-store : Services



Exemple d'un e-store : Couches



Avant-propos

Ce cahier des charges définit de manière précise les objectifs, le périmètre, l'architecture, les technologies, les fonctionnalités, les contraintes et les livrables du mini-projet E-Store demandé dans le cadre du module. Il doit servir de document de référence pendant toute la durée du projet.

Le sujet a été construit en cohérence avec les notions étudiées dans le cours : architecture logicielle, découpage en couches, décomposition par domaines, développement de services avec Spring Boot, persistance avec Spring Data JPA, exposition d'API REST, intégration de MongoDB pour les données documentaires et consommation des services depuis un frontend Angular.

L'objectif n'est pas seulement de produire un site marchand simplifié. Le véritable objectif pédagogique est d'apprendre à structurer une application comme un ingénieur logiciel, avec une séparation claire des responsabilités et une organisation de code maintenable.

Comment lire ce document

Les parties 1 à 6 expliquent le contexte, les objectifs et l'architecture.

Les parties 7 à 14 détaillent les fonctionnalités, les données et les exigences techniques.

Les parties 15 à 20 précisent les livrables, le planning, l'évaluation et la maintenance.

Les annexes donnent des conseils de mise en œuvre, des structures de projet et des recommandations pratiques.

Sommaire thématique

1. Contexte et justification du mini-projet
2. Objectifs pédagogiques et compétences visées
3. Présentation du sujet E-Store
4. Lecture et explication des schémas du cours
5. Architecture générale imposée
6. Découpage fonctionnel par domaines

7. Exigences fonctionnelles détaillées
8. Modèle de données relationnel et documentaire
9. Exigences backend Spring Boot et Spring Data JPA
10. Exigences frontend Angular
11. API REST à réaliser
12. Contraintes de qualité, sécurité et tests
13. Livrables, planning et organisation du groupe
14. Barème d'évaluation et consignes de soutenance
15. Annexes pratiques pour démarrer rapidement

1. Contexte et justification du mini-projet

Le mini-projet proposé est une plateforme E-Store, c'est-à-dire une boutique en ligne simplifiée. Le choix de ce sujet n'est pas arbitraire. Il permet de manipuler à la fois des notions de gestion des utilisateurs, de consultation d'un catalogue, de gestion de stock, de panier, de commande et de traçabilité. Autrement dit, il met les étudiants face à un cas réaliste d'intégration de plusieurs préoccupations logicielles dans une seule application cohérente.

Dans le cours, le mini-projet E-Store apparaît comme un exemple pédagogique d'architecture logicielle. Les schémas montrent à la fois une vue par couches, une vue par domaines et une vue par services. Cela signifie que le sujet est particulièrement adapté pour comprendre qu'une application n'est pas un ensemble de pages isolées, mais un système organisé dans lequel les responsabilités sont distribuées avec méthode.

Le projet s'inscrit également dans la logique d'un développement full stack moderne. Le backend est développé avec Spring Boot pour gérer la logique métier et exposer les services REST. La persistance relationnelle s'appuie sur Spring Data JPA, tandis que MongoDB est utilisée pour les besoins documentaires ou semi-structurés. Enfin, Angular prend en charge la couche présentation côté client.

Ce mini-projet permet donc d'articuler plusieurs objectifs du module : structurer un code propre, comprendre l'intérêt d'un framework, manipuler des repositories JPA, raisonner sur le découpage applicatif, consommer des API REST et réaliser une interface moderne.

En pratique, le projet doit être pensé comme un travail d'ingénierie logicielle. La réussite ne dépend pas uniquement du nombre de pages réalisées ou du style graphique. Elle dépend surtout de la qualité de l'analyse, de la cohérence de l'architecture, de la clarté du code, de la justesse du modèle de données et de la capacité du groupe à démontrer que chaque décision technique répond à un besoin précis.

2. Objectifs pédagogiques et compétences visées

À la fin du mini-projet, l'étudiant devra être capable d'analyser un besoin, d'identifier les modules nécessaires, de proposer un découpage en couches et en domaines, puis de traduire cette analyse sous forme d'une application fonctionnelle.

Sur le plan technique, le projet vise l'acquisition ou le renforcement des compétences suivantes : création d'un projet Spring Boot avec Maven, définition d'entités JPA et de leurs relations, écriture de repositories et de services, création de contrôleurs REST, configuration de la base relationnelle, utilisation de MongoDB, création d'un frontend Angular, routage, formulaires, consommation d'API et gestion d'état simple côté client.

Sur le plan méthodologique, les étudiants devront apprendre à répartir les tâches, documenter leurs choix, tester progressivement les composants, versionner le code et préparer une démonstration orale structurée. Le mini-projet constitue donc aussi un entraînement à la collaboration technique et à la maintenance.

Sur le plan conceptuel, les étudiants devront comprendre qu'une architecture solide réduit la complexité. Une bonne architecture améliore la maintenabilité, facilite les évolutions et réduit les erreurs. Ce point est essentiel car il relie directement la théorie du cours à la pratique de développement.

Enfin, le projet doit aider l'étudiant à passer d'une logique d'exécution de tutoriels à une logique de conception. Le cahier des charges ne demande pas seulement d'imiter un exemple, mais de construire une application cohérente à partir d'un besoin, d'une architecture et de contraintes explicites.

3. Présentation du sujet E-Store

L'application représente une petite boutique en ligne dans laquelle un utilisateur peut créer un compte, se connecter, consulter un catalogue de produits, effectuer une recherche, afficher la fiche détaillée d'un produit, ajouter des articles au panier puis confirmer une commande.

Du côté de l'administrateur ou d'un rôle de gestion, le système peut permettre l'ajout de produits, l'actualisation du stock, la consultation des commandes ou l'analyse des données documentaires stockées dans MongoDB. Cette partie peut être partielle dans la version minimale et enrichie dans la version bonus.

Le sujet s'appuie sur cinq domaines fonctionnels clairement visibles dans les schémas du cours : Customer, Catalog, Inventory, Shopping et Billing. Chacun de ces domaines correspond à un sous-ensemble d'objets métier, de services et d'interactions côté interface.

Le domaine Customer gère la création de compte, la connexion et le profil utilisateur. Le domaine Catalog gère les produits, catégories et recherches. Le domaine Inventory gère le stock. Le domaine Shopping s'occupe du panier et du parcours d'achat. Le domaine Billing gère la commande, son enregistrement et son suivi simplifié.

L'application doit rester simple, mais elle doit être pensée comme un système complet. Chaque domaine a une responsabilité bien définie et doit communiquer avec les autres de manière claire, par exemple lorsqu'une commande validée déclenche une vérification du stock et un enregistrement dans l'historique des commandes.

4. Lecture et explication des schémas du cours

Les schémas fournis dans le cours ne sont pas de simples illustrations décoratives. Ils résument la logique d'architecture du mini-projet. Le premier schéma, centré sur les couches, montre que l'application doit être organisée selon une hiérarchie claire : couche présentation, couche logique métier et couche accès aux données.

La couche présentation correspond à tout ce que l'utilisateur voit ou manipule directement : pages, formulaires, routes Angular, composants, listes de produits, fiches détail, panier et écrans de commande. C'est la partie visible du système.

La couche logique métier correspond aux règles de gestion. Elle ne doit pas contenir de code d'interface ni de code SQL direct. C'est dans cette couche que l'on vérifie la validité des données, que l'on calcule un total de panier, que l'on contrôle le stock disponible ou que l'on prépare une commande.

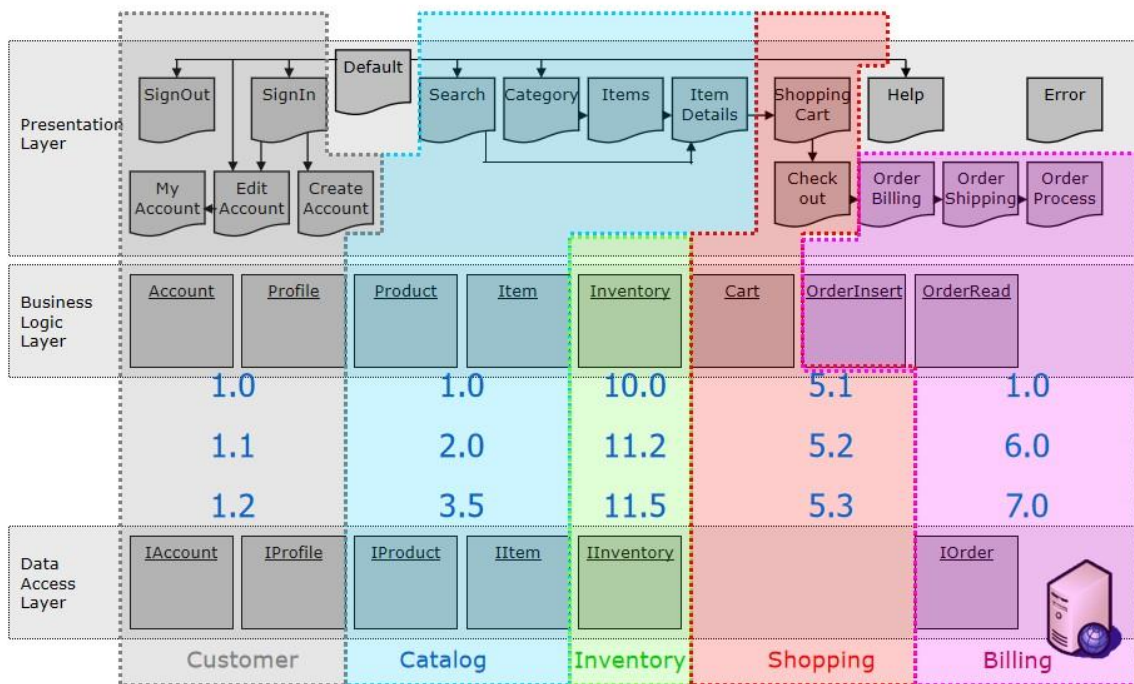
La couche accès aux données constitue l'interface entre la logique métier et les bases de données. Avec Spring Data JPA, elle contient les repositories relationnels. Avec MongoDB, elle contient les repositories documentaires. Son rôle est de persister et de

recupérer l'information sans polluer la couche métier avec des détails techniques de stockage.

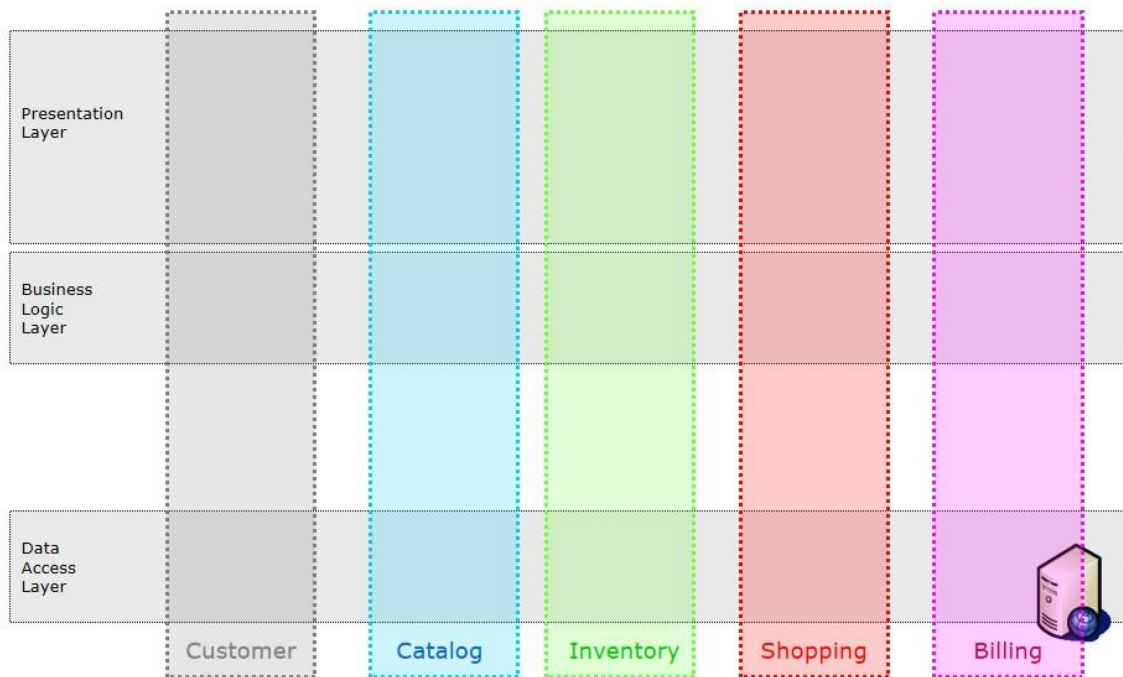
Le schéma par domaines apporte une autre lecture. Il montre que l'on peut découper la même application non plus seulement en niveaux techniques, mais aussi en grands sous-systèmes fonctionnels : Customer, Catalog, Inventory, Shopping et Billing. Cette vue est très importante car elle permet d'organiser les packages, les services, les entités et les composants Angular de manière plus professionnelle.

Le schéma par services ajoute encore une dimension. Il montre que l'application peut aussi être pensée en services orientés métier : Manage Customer, Show Catalog, Make Inventory, Shop et Bill. Cette vue est utile pour concevoir les API REST, les cas d'usage et, plus tard, une éventuelle évolution vers des microservices.

Exemple d'un e-store : Domaines



Exemple d'un e-store : Domaines



5. Architecture générale imposée

Le projet doit impérativement respecter une architecture à plusieurs couches. L'objectif n'est pas de mélanger les responsabilités dans un seul package ou un seul composant, mais de structurer l'application de façon lisible et évolutive.

La couche présentation sera réalisée avec Angular. Elle inclut les composants, le routage, les templates HTML, les services Angular de communication avec le backend, la validation des formulaires et la navigation générale.

La couche métier sera réalisée avec Spring Boot. Elle comportera les contrôleurs REST, les services métier, les DTO éventuels, les règles de validation métier, la gestion des exceptions et l'orchestration des traitements.

La couche accès aux données s'appuiera sur deux mécanismes. Le premier est relationnel avec JPA pour les données fortement structurées. Le second est documentaire avec MongoDB pour les données plus souples, telles que les avis, historiques de recherche ou journaux applicatifs simplifiés.

Cette architecture doit apparaître de manière claire aussi bien dans le code que dans la documentation. Le rapport et la soutenance devront montrer comment les étudiants ont traduit cette organisation dans leur projet.

Toute déviation importante par rapport à cette structure devra être justifiée. Un projet qui fonctionne mais qui ne respecte pas l'architecture imposée ne pourra pas être considéré comme excellent.

6. Découpage fonctionnel par domaines

Le domaine Customer couvre l'identité de l'utilisateur et son cycle d'interaction avec le système. Il comprend au minimum l'inscription, la connexion, la déconnexion, la consultation du profil et la modification des informations personnelles.

Le domaine Catalog couvre la gestion et l'affichage des produits. Il doit inclure les catégories, la liste des produits, la recherche, le filtrage et l'affichage détaillé d'une fiche produit.

Le domaine Inventory gère l'état du stock. Son rôle est de fournir à la logique métier les informations nécessaires pour savoir si un article peut être ajouté au panier ou confirmé dans une commande.

Le domaine Shopping gère le panier. Il comprend l'ajout d'articles, la suppression, la modification de quantité, le calcul du montant total et la préparation du checkout.

Le domaine Billing gère la partie commande. Il correspond à l'enregistrement d'une commande validée, à son historique, à un état éventuel de traitement et à une éventuelle préparation de facturation simplifiée.

Chaque domaine doit idéalement disposer de ses propres entités principales, services, contrôleurs, repositories et composants Angular, même si certains objets sont transversaux ou partagés.

Domaine	Responsabilité principale	Objets / composants	Résultats attendus
Customer	Gérer le compte et le profil	User, Profile, Auth	Inscription, connexion, profil
Catalog	Exposer les produits	Category, Product	Liste, détail, recherche

Inventory	Contrôler la disponibilité	Inventory	Vérification et mise à jour du stock
Shopping	Gérer le panier	Cart, CartItem	Ajout, suppression, total
Billing	Enregistrer la commande	Order, OrderItem	Validation et historique

7. Exigences fonctionnelles détaillées

L'application doit proposer une page d'accueil claire, à partir de laquelle l'utilisateur peut accéder au catalogue, à l'inscription ou à la connexion. La navigation doit être cohérente et accessible depuis un menu supérieur ou latéral.

La fonctionnalité d'inscription doit permettre à un nouvel utilisateur de créer un compte à l'aide d'un formulaire validé. L'adresse e-mail doit être unique. Les champs obligatoires doivent être contrôlés côté client et côté serveur.

La fonctionnalité de connexion doit permettre l'authentification d'un utilisateur existant. Une gestion simple de session côté frontend est acceptable dans la version minimale. L'usage de JWT est un bonus fortement valorisé.

Le catalogue doit permettre l'affichage des produits sous forme de cartes ou de lignes, avec au minimum le nom, une image, le prix, la catégorie et un bouton d'accès au détail. Une recherche par mot-clé doit être disponible. Un filtrage par catégorie est vivement recommandé.

La fiche produit doit présenter les détails d'un article : description, prix, disponibilité, image, catégorie, avis éventuels ou informations complémentaires. L'utilisateur doit pouvoir ajouter le produit au panier si le stock est suffisant.

Le panier doit afficher les articles sélectionnés, leurs quantités, le prix unitaire, le sous-total par ligne et le total général. L'utilisateur doit pouvoir modifier la quantité ou supprimer un article. Un bouton de validation doit lancer le processus de commande.

La commande doit enregistrer l'ensemble des lignes du panier, le total, la date et le lien avec l'utilisateur. Le stock doit être mis à jour. Une page ou un message de

confirmation doit être affiché. L'utilisateur doit pouvoir consulter ses anciennes commandes.

La partie MongoDB doit être concrète. Le groupe choisira soit la gestion des avis produits, soit l'historique de recherche, soit un module documentaire équivalent approuvé par l'enseignant. Ce module ne doit pas être artificiel : il doit être intégré au parcours applicatif.

8. Exigences non fonctionnelles et qualité attendue

L'application doit être structurée de façon lisible. Les noms des classes, méthodes, packages, composants et variables doivent être explicites. Le code doit éviter la duplication excessive et respecter autant que possible les principes de responsabilité unique.

L'interface utilisateur doit être compréhensible, propre et homogène. Le rendu n'a pas besoin d'être luxueux, mais il doit être sérieux, responsive et agréable à utiliser. Des messages de succès et d'erreur clairs sont attendus.

Le projet doit être stable. Les erreurs classiques doivent être gérées proprement, par exemple lorsqu'un produit n'existe pas, lorsqu'un stock est insuffisant ou lorsqu'un formulaire contient des données invalides.

Les données doivent être cohérentes. Une commande ne doit pas pouvoir être validée si le panier est vide ou si les quantités dépassent le stock. Les champs obligatoires doivent être vérifiés. Les dates, prix et quantités doivent être manipulés proprement.

Les équipes sont également invitées à tenir compte de la sécurité de base : validation serveur, non-exposition d'informations sensibles inutiles, hachage du mot de passe si un vrai module d'authentification est mis en place, séparation des accès côté frontend.

Un effort de test est attendu. Il peut s'agir de tests unitaires simples, de tests d'API avec Postman, de scénarios fonctionnels manuels ou d'une combinaison des trois.

9. Modèle de données relationnel avec JPA

La partie relationnelle doit couvrir les données transactionnelles principales. Le modèle minimal recommandé comprend les entités User, Profile, Category, Product, Inventory, Cart, CartItem, Order et OrderItem.

L'entité User représente l'utilisateur authentifié du système. Elle contient les informations d'identité et de connexion. L'entité Profile complète le profil avec des données telles que téléphone, adresse, ville ou pays.

L'entité Category structure les produits. L'entité Product représente le produit métier. L'entité Inventory permet de gérer la quantité disponible. Selon le choix de conception, le stock peut être un champ du produit ou une entité distincte. Une entité distincte est préférable car elle clarifie la responsabilité métier.

Le panier peut être modélisé avec Cart et CartItem. Cart représente le panier d'un utilisateur et CartItem représente une ligne du panier, c'est-à-dire un produit avec une quantité et éventuellement un prix unitaire figé au moment de l'ajout.

La commande doit être modélisée avec Order et OrderItem. Order représente l'entête de commande avec la date, le total et le statut. OrderItem représente les lignes de commande. Cette séparation est indispensable pour bien conserver l'historique d'achat.

Les relations JPA devront être choisies avec soin. On attend en général des associations de type one-to-one entre User et Profile, one-to-many entre Order et OrderItem, one-to-many entre Cart et CartItem, many-to-one entre Product et Category, et des liens clairs entre Order, User et Product.

Entité	Clé	Attributs essentiels	Relations principales
User	id	firstName, lastName, email, password	1-1 avec Profile, 1-N avec Order
Profile	id	phone, address, city, country	1-1 avec User
Category	id	name, description	1-N avec Product
Product	id	name, price, imageUrl, description	N-1 avec Category, 1-1 ou 1-N avec Inventory
Inventory	id	quantity	lié à Product

Cart	id	createdAt	N-1 avec User, 1-N avec CartItem
CartItem	id	quantity, unitPrice	N-1 avec Cart et Product
Order	id	orderDate, totalAmount, status	N-1 avec User, 1-N avec OrderItem
OrderItem	id	quantity, unitPrice	N-1 avec Order et Product

10. Utilisation de MongoDB

MongoDB doit être utilisée pour un cas d'usage pertinent. Le plus pédagogique consiste à gérer des avis sur les produits. Chaque avis peut contenir un identifiant, l'identifiant du produit, l'identifiant de l'utilisateur, le nom de l'auteur, une note, un commentaire et une date.

Ce choix présente plusieurs avantages. Les avis sont naturellement des données semi-structurées, parfois évolutives, et ne nécessitent pas toujours le même degré de normalisation qu'une commande ou un stock. Cela permet d'illustrer l'intérêt du stockage documentaire en complément du relationnel.

Une autre possibilité consiste à stocker l'historique de recherche. Dans ce cas, chaque document contient l'utilisateur, le mot-clé recherché et l'horodatage. Cette approche est plus légère mais reste acceptable si elle est bien intégrée à l'interface et à l'analyse métier.

Quel que soit le choix retenu, MongoDB ne doit pas être utilisée juste pour dire qu'elle a été utilisée. Il faut un écran, une API, un repository et un scénario démontrable. On doit pouvoir voir concrètement le rôle de MongoDB dans l'application.

Le rapport devra expliquer pourquoi certaines données ont été stockées dans la base relationnelle et d'autres dans MongoDB. Cette justification fait partie de la qualité d'analyse attendue.

11. Exigences backend Spring Boot

Le backend doit être organisé de manière claire, idéalement par domaines fonctionnels. Par exemple, chaque domaine peut posséder ses sous-packages entity, repository, service, controller, dto et mapper. Une autre approche acceptable consiste à séparer d'abord par couches puis à garder une cohérence par domaine. Dans tous les cas, le code doit rester lisible.

Les contrôleurs REST ne doivent pas contenir de logique métier complexe. Leur rôle principal est de recevoir les requêtes HTTP, de déléguer au service adéquat et de construire la réponse. Les services doivent porter les règles métier. Les repositories doivent rester concentrés sur l'accès aux données.

Les dépendances Maven attendues incluent au minimum spring-boot-starter-web, spring-boot-starter-data-jpa, le driver de la base relationnelle, spring-boot-starter-validation et spring-boot-starter-data-mongodb. L'usage de Lombok est possible, mais il doit être compris et non utilisé aveuglément.

Une gestion propre des exceptions est fortement recommandée. Un contrôleur d'exception global peut être mis en place afin de renvoyer des messages d'erreur cohérents. Par exemple : ressource introuvable, validation échouée, stock insuffisant ou opération interdite.

Le backend doit exposer des endpoints REST cohérents, avec des URI lisibles et un usage correct des verbes HTTP. On attend une séparation raisonnable entre GET pour la consultation, POST pour la création, PUT ou PATCH pour la mise à jour et DELETE pour la suppression.

Le projet doit être exécutable facilement. Un fichier application.properties ou application.yml doit être fourni avec une configuration claire de la base relationnelle et de MongoDB. Des données d'initialisation ou des comptes de test sont appréciés.

Verbe	URI	Domaine	Description
POST	/api/auth/register	Customer	Créer un compte utilisateur
POST	/api/auth/login	Customer	Authentifier l'utilisateur
GET	/api/products	Catalog	Lister les produits

GET	/api/products/{id}	Catalog	Afficher un produit
GET	/api/categories	Catalog	Lister les catégories
GET	/api/cart/{userId}	Shopping	Consulter le panier
POST	/api/cart/add	Shopping	Ajouter un produit au panier
PUT	/api/cart/update	Shopping	Modifier la quantité
DELETE	/api/cart/remove/{itemId}	Shopping	Supprimer une ligne
POST	/api/orders	Billing	Valider une commande
GET	/api/orders/user/{userId}	Billing	Historique des commandes
POST	/api/reviews	MongoDB	Créer un avis produit
GET	/api/reviews/product/{productId}	MongoDB	Afficher les avis d'un produit

12. Exigences frontend Angular

Le frontend Angular doit refléter la structure fonctionnelle du projet. Il est conseillé de créer des modules ou dossiers correspondant aux grands domaines : auth, catalog, cart, orders, profile, shared, core, etc.

Chaque composant doit avoir un rôle clair. On attend par exemple des composants pour la liste des produits, la fiche produit, le panier, la connexion, l'inscription, le profil et l'historique des commandes. Les services Angular doivent être responsables des appels HTTP vers le backend.

Le routage doit être propre et explicite. Des routes distinctes doivent exister pour le catalogue, le détail d'un produit, le panier, la page de connexion, l'inscription, le profil et l'historique des commandes.

Les formulaires Angular doivent être validés. Les messages d'erreur doivent apparaître à proximité des champs concernés. Le projet sera mieux évalué si les validations client et serveur sont cohérentes.

Une charte visuelle simple mais propre est attendue. Bootstrap ou Angular Material peuvent être utilisés. Le design doit rester au service de la compréhension. Il n'est pas nécessaire de surcharger l'interface d'effets visuels.

L'intégration avec les API REST doit être démontrée. Les données affichées doivent provenir du backend réel et non de tableaux statiques codés en dur, sauf éventuellement au tout début de la phase de prototypage.

13. Spécification des API REST

Pour la partie authentification, un endpoint d'inscription et un endpoint de connexion sont attendus. Même dans une version simple, il faut distinguer les opérations de création de compte et d'authentification.

Pour les utilisateurs, l'API doit permettre de consulter un profil et de le mettre à jour. Pour les catégories et les produits, il faut des endpoints de consultation, et idéalement d'administration si le groupe implémente une gestion back-office minimale.

Pour le panier, l'API doit offrir au minimum : récupération du panier d'un utilisateur, ajout d'un produit, mise à jour de quantité et suppression d'une ligne. Pour les commandes, il faut un endpoint de validation et un endpoint de consultation de l'historique par utilisateur.

Pour MongoDB, des endpoints doivent exister selon le choix fonctionnel. Par exemple, pour les avis : création d'un avis et récupération des avis d'un produit. Pour l'historique de recherche : enregistrement d'une recherche et récupération de l'historique par utilisateur.

Les réponses JSON doivent être compréhensibles. Les codes de statut HTTP doivent être utilisés proprement. Une ressource non trouvée doit renvoyer un 404. Une validation incorrecte doit renvoyer un code adapté, par exemple 400. Une création réussie doit idéalement renvoyer un 201.

L'équipe est libre de définir des DTO pour éviter d'exposer directement certaines entités. Cette pratique est encouragée si elle est maîtrisée.

14. Organisation recommandée du code

Une structure backend possible est la suivante : un package racine estore contenant customer, catalog, inventory, shopping, billing, shared et config. Chaque domaine contient ses entités, services, repositories et contrôleurs.

Une structure frontend possible est la suivante : app/core pour les services globaux et l'infrastructure, app/shared pour les composants réutilisables, app/features/auth, app/features/catalog, app/features/cart, app/features/orders et app/features/profile.

Cette organisation doit être documentée dans le rapport. Les étudiants devront montrer qu'ils ont compris le lien entre les vues du schéma du cours et leur propre structure de code.

Il est recommandé de centraliser les constantes utiles, les classes de réponse standardisées et certains utilitaires. En revanche, il faut éviter les classes fourre-tout qui deviennent un espace de mélange de responsabilités.

Les noms doivent être professionnels. Il faut éviter les abréviations non explicites ou les packages désordonnés. Un projet bien nommé est souvent un projet mieux pensé.

15. Cas d'usage détaillés

Cas d'usage 1 : inscription. Un visiteur accède au formulaire d'inscription, saisit ses informations, valide le formulaire, le backend vérifie l'unicité de l'e-mail puis crée l'utilisateur et éventuellement son profil de base.

Cas d'usage 2 : connexion. Un utilisateur déjà inscrit saisit ses identifiants. Le backend les vérifie puis renvoie une réponse indiquant le succès ou l'échec. Le frontend stocke l'état de connexion selon la stratégie choisie par l'équipe.

Cas d'usage 3 : recherche et consultation du catalogue. L'utilisateur accède à la liste des produits, filtre éventuellement par catégorie, consulte une fiche détail et visualise la disponibilité.

Cas d'usage 4 : ajout au panier. Depuis la fiche produit ou la liste, l'utilisateur ajoute un article à son panier. Le backend vérifie le stock et crée ou met à jour la ligne de panier.

Cas d'usage 5 : validation de commande. L'utilisateur ouvre son panier, vérifie le total, lance la validation. Le backend crée une commande, transfère les lignes du panier vers

les lignes de commande, met à jour le stock et vide le panier ou le marque comme traité.

Cas d'usage 6 : consultation historique. L'utilisateur accède à la liste de ses commandes passées. Il peut visualiser le détail d'une commande, sa date, son total et son état.

16. Jeux de données et scénarios de démonstration

Chaque groupe doit préparer un minimum de données de démonstration. Cela signifie plusieurs catégories, plusieurs produits, plusieurs comptes utilisateurs et au moins quelques documents MongoDB si le module documentaire est implémenté.

La démonstration doit montrer un parcours complet : inscription ou connexion, consultation du catalogue, ajout au panier, validation de commande et consultation de l'historique. Si des avis sont implémentés, il faut également montrer leur création et leur affichage.

Les produits choisis peuvent appartenir à n'importe quel secteur : informatique, livres, sport, vêtements, fournitures, etc. L'important est que les données soient cohérentes et suffisamment variées pour justifier l'existence des catégories, du stock et des opérations de panier.

Les étudiants doivent préparer un scénario sans erreur et au moins un scénario d'erreur contrôlée, par exemple tentative de commande d'une quantité supérieure au stock, ou validation d'un formulaire incomplet. Cela montrera la robustesse du projet.

L'usage de scripts SQL de remplissage, de CommandLineRunner ou de scripts MongoDB est encouragé pour accélérer la démonstration.

17. Livrables obligatoires

Le premier livrable est le code source complet du backend Spring Boot. Il doit être fourni sous forme de projet Maven ouvrable et exécutable. Les dépendances doivent être correctement déclarées.

Le deuxième livrable est le code source complet du frontend Angular. Le projet doit pouvoir être lancé avec les commandes standards de l'écosystème Angular. Les dépendances doivent être installables sans modification majeure.

Le troisième livrable est la documentation technique, sous forme d'un rapport PDF. Ce rapport doit présenter le contexte, l'architecture, le modèle de données, les API, les écrans, les choix techniques, les difficultés et les perspectives d'amélioration.

Le quatrième livrable comprend les scripts ou instructions nécessaires pour créer la base relationnelle et configurer MongoDB. Un README clair est obligatoire. Il doit expliquer comment exécuter le backend, le frontend et les bases.

Le cinquième livrable est la présentation orale. Les étudiants doivent être capables de justifier leur architecture, de montrer les principales fonctionnalités et de répondre à des questions techniques simples.

Tout projet non documenté, non exécutable ou incomplet pourra être fortement pénalisé, même si certaines parties du code sont intéressantes.

#	Livrable	Contenu attendu
1	Backend Spring Boot	Code source Maven, configuration, entités, services, API
2	Frontend Angular	Composants, routing, formulaires, services HTTP
3	Scripts BD	Création SQL et configuration MongoDB
4	README	Étapes de lancement et comptes de test
5	Rapport PDF	Architecture, données, captures, choix techniques
6	Jeu de données	Données de démonstration prêtes à l'emploi
7	Présentation orale	Démonstration fonctionnelle et justification technique

18. Planning conseillé et méthode de travail

Semaine 1 : lecture du cahier des charges, compréhension des schémas, choix du cas MongoDB, répartition initiale des tâches, conception du modèle de données et structure des projets backend et frontend.

Semaine 2 : implémentation des entités, repositories et services principaux côté backend. Mise en place de la base relationnelle. Création des premiers endpoints REST et tests avec Postman.

Semaine 3 : développement du frontend Angular, intégration des écrans principaux, connexion aux API, mise en place du panier et du parcours de commande.

Semaine 4 : finalisation du module MongoDB, amélioration de l'interface, gestion des erreurs, scénarios de test, rédaction du rapport, préparation de la soutenance.

Le groupe est encouragé à utiliser Git et à définir un mode de collaboration simple. Chaque membre doit avoir une responsabilité claire, mais tout le monde doit être en mesure d'expliquer l'ensemble du projet au moment de la soutenance.

Il est conseillé de procéder par incréments : une fonctionnalité simple mais complète vaut mieux qu'un grand nombre d'écrans partiellement reliés.

19. Barème d'évaluation proposé

Analyse du besoin et respect global du cahier des charges : 3 points.

Architecture logicielle et qualité de structuration du projet : 4 points.

Backend Spring Boot et Spring Data JPA : 4 points.

Intégration cohérente de MongoDB : 2 points.

Frontend Angular et expérience utilisateur : 3 points.

Qualité fonctionnelle et robustesse globale : 2 points.

Rapport, README et qualité de la soutenance : 2 points.

Un bonus peut être accordé pour l'intégration de Spring Security, JWT, pagination, rôle administrateur, tests automatiques ou améliorations particulièrement pertinentes.

20. Conseils pour la soutenance

La soutenance doit commencer par une présentation brève du besoin et de l'architecture. Les étudiants doivent montrer qu'ils ont compris le lien entre les schémas du cours et leur implémentation réelle.

Il est recommandé d'expliquer d'abord la vue en couches, puis la vue par domaines, puis la structure du code. Ensuite, l'équipe peut passer à la démonstration fonctionnelle.

La démonstration doit être fluide. Il faut éviter de chercher des données au dernier moment ou de corriger le code en direct. Les jeux de données doivent être préparés.

Chaque étudiant doit pouvoir répondre à des questions simples : pourquoi avez-vous choisi cette structure, pourquoi JPA ici, pourquoi MongoDB là, où se trouve la logique métier, comment la commande met-elle à jour le stock, etc.

Une bonne soutenance ne repose pas sur un discours long, mais sur une démonstration cohérente, un vocabulaire précis et la capacité à justifier les choix techniques.

Annexe A - Structure recommandée du backend

src/main/java/com/estore

- | - customer
 - | | - controller
 - | | - service
 - | | - repository
 - | | - entity
 - | | - dto
- | - catalog
- | - inventory
- | - shopping
- | - billing
- | - shared
- | - config
- | - exception

Cette structure n'est pas la seule possible, mais elle est fortement recommandée car elle épouse le découpage fonctionnel du mini-projet et facilite la lisibilité.

Annexe B - Structure recommandée du frontend Angular

src/app

- | - core
- | - shared
- | - features/auth
- | - features/catalog
- | - features/cart
- | - features/orders
- | - features/profile
- | - app-routing.module.ts
- | - app.module.ts

Le frontend peut naturellement évoluer selon votre niveau, mais la cohérence de l’organisation doit rester visible.

Annexe C - Exemple de planning hebdomadaire

Semaine	Objectif principal	Travaux attendus	Sortie attendue
1	Conception	Analyse, architecture, MCD, setup projets	Schéma validé et projets initialisés
2	Backend	Entités, repositories, services, API	Premières API testées avec Postman
3	Frontend	Écrans principaux et intégration	Catalogue, login, panier
4	Finalisation	Commande, MongoDB, tests, rapport	Projet démontrable et documenté

Annexe D - Conseils pratiques

- Commencez par un parcours minimum viable : login, catalogue, panier, commande.
- Testez chaque endpoint dès sa création avec Postman.
- N’attendez pas la fin pour brancher Angular au backend.
- Gardez des jeux de données cohérents et réalistes.
- Ne mélangez pas logique métier, accès aux données et logique de présentation.
- Préparez votre soutenance plusieurs jours avant la date de remise.

Bon courage